

Relatorio: Desenvolvimento de um compilador de operações matemáticas na linguagem Python

Pablio Wyllams Tavares Barbosa - 01633453

Flavio Miguel Guilherme Dos Santos - 01647888

Cicero Barros Da Silva - 01638161

Edllan Kauã Da Silva Oliveira - 01611518

Eduardo Leandro Santos - 01670259

11 de Junho de 2026

1 Introdução e Arquitetura Global

A engenharia de compiladores estuda as técnicas necessárias para traduzir ou interpretar linguagens de programação de alto nível. O código analisado propõe um interpretador de expressões aritméticas baseado em uma Gramática Livre de Contexto (GLC) com prioridades de operadores e associatividade explícitas.

O fluxo de dados do compilador segue uma estrutura linear dividida em:

1. **Cadeia de Entrada:** Código fonte bruto fornecido pelo usuário.
2. **Analizador Léxico (Tokenizer):** Quebra a cadeia de texto em tokens tipados baseados em expressões regulares.
3. **Analizador Sintático (Parser):** Consome os tokens baseando-se em regras gramaticais e calcula de forma imediata o resultado semântico.

2 Análise Léxica (Lexer)

O processo de tokenização varre a string de entrada comparando-a com uma lista de padrões descritos via expressões regulares (Regex).

2.1 Tabela de Especificação de Tokens

Abaixo são mapeados os tipos de tokens aceitos pelo motor do *Lexer*:

Tabela 1: Especificação de Expressões Regulares dos Tokens

Token Class	Expressão Regular	Descrição
NUMBER	<code>\d+(\.\d+)?</code>	Números inteiros ou de ponto flutuante
IDENT	<code>[A-Za-z_][A-Za-z0-9_]*</code>	Variáveis e Identificadores
POW	<code>**\^</code>	Operadores de Exponenciação
PLUS / MINUS	<code>\+ / -</code>	Adição e Subtração
MUL / DIV	<code>* / /</code>	Multiplicação e Divisão
ASSIGN	<code>=</code>	Atribuição de variáveis
LPAREN / RPAREN	<code>\(/ \)</code>	Delimitadores de Parênteses
LBRACK / RBRACK	<code>\[/ \]</code>	Delimitadores de Colchetes
LBRACE / RBRACE	<code>\{ / \}</code>	Delimitadores de Chaves

3 Análise Sintática e Gramática Formal

O *Parser* utiliza uma abordagem Descendente Recursiva (*Recursive Descent Parser*), onde cada nível de precedência matemática é isolado em um método específico (`expr`, `term`, `power`, `factor`).

3.1 Gramática na Notação EBNF

A gramática livre de contexto pode ser descrita formalmente como:

$$\begin{aligned} \text{Parse} &\rightarrow \text{IDENT} = \text{Expr} \mid \text{Expr} \\ \text{Expr} &\rightarrow \text{Term} \{ (+ \mid -) \text{Term} \} \\ \text{Term} &\rightarrow \text{Power} \{ (* \mid /) \text{Power} \} \\ \text{Power} &\rightarrow \text{Factor} [** \mid \wedge \text{Power}] \\ \text{Factor} &\rightarrow \text{NUMBER} \mid \text{IDENT} \mid (\text{Expr}) \mid [\text{Expr}] \mid \{ \text{Expr} \} \end{aligned}$$

A regra *Power* introduz a associatividade à direita, permitindo resolver corretamente cálculos encadeados como $2 \wedge 3 \wedge 2 = 2^{(3^2)} = 512$.

4 Implementação em Python

Abaixo encontra-se o código fonte unificado que serve como fundação estrutural deste sistema compilador:

```
1 import re
2
3 ENV = {}
4
5 def compiler(code):
6     try:
7         tokens = tokenize(code)
8         parser = Parser(tokens, ENV)
9         return parser.parse()
10    except Exception as e:
11        return f"Erro: {e}"
12
13 TOKEN_SPECIFICATION = [
14     ('NUMBER', r'\d+(\.\d+)?'),
15     ('IDENT', r'[A-Za-z_][A-Za-z0-9_]*'),
16     ('POW', r'\*\*|\^'),
17     ('PLUS', r'\+'),
18     ('MINUS', r'\-'),
19     ('MUL', r'\*'),
20     ('DIV', r'/'),
21     ('ASSIGN', r '='),
22     ('LPAREN', r'\('),
23     ('RPAREN', r'\)'),
24     ('LBRACK', r'\['),
25     ('RBRACK', r'\]'),
26     ('LBRACE', r'\{'),
27     ('RBRACE', r'\}'),
28     ('SKIP', r'[\t\n]+'),
29     ('MISMATCH', r'.'),
30 ]
31
32 def tokenize(code):
33     tok_regex = '|'.join(f'(?P<{name}>{regex})' for name, regex in
34                             TOKEN_SPECIFICATION)
35     tokens = []
36     for mo in re.finditer(tok_regex, code):
```

```

36     kind = mo.lastgroup
37     value = mo.group()
38     if kind == 'NUMBER':
39         tokens.append(('NUMBER', float(value)))
40     elif kind == 'IDENT':
41         tokens.append(('IDENT', value))
42     elif kind == 'SKIP':
43         continue
44     elif kind == 'MISMATCH':
45         raise SyntaxError(f'Caractere inesperado: {value}')
46     else:
47         tokens.append((kind, value))
48 tokens.append(('EOF', None))
49 return tokens
50
51 class Parser:
52     def __init__(self, tokens, env=None):
53         self.tokens = tokens
54         self.pos = 0
55         self.current_token = self.tokens[self.pos]
56         self.env = env if env is not None else {}
57
58     def error(self, message="Erro de sintaxe"):
59         raise SyntaxError(message)
60
61     def advance(self):
62         self.pos += 1
63         if self.pos < len(self.tokens):
64             self.current_token = self.tokens[self.pos]
65
66     def eat(self, token_type):
67         if self.current_token[0] == token_type:
68             self.advance()
69         else:
70             self.error(f"Esperado {token_type}, encontrado {self.
current_token[0]}")
71
72     def factor(self):
73         token = self.current_token
74         if token[0] == 'NUMBER':
75             self.eat('NUMBER')
76             return token[1]
77         elif token[0] == 'IDENT':
78             name = token[1]
79             self.eat('IDENT')
80             if name in self.env:
81                 return self.env[name]
82             self.error(f"Variavel nao definida: {name}")
83         elif token[0] == 'LPAREN':
84             self.eat('LPAREN')
85             result = self.expr()
86             self.eat('RPAREN')
87             return result
88         elif token[0] == 'LBRACK':
89             self.eat('LBRACK')
90             result = self.expr()
91             self.eat('RBRACK')
92             return result

```

```

93         elif token[0] == 'LBRACE':
94             self.eat('LBRACE')
95             result = self.expr()
96             self.eat('RBRACE')
97             return result
98         self.error(f"Fator invalido: {token[1]}")
99
100     def term(self):
101         node = self.power()
102         while self.current_token[0] in ('MUL', 'DIV'):
103             op = self.current_token[0]
104             self.advance()
105             if op == 'MUL':
106                 node = node * self.power()
107             elif op == 'DIV':
108                 denom = self.power()
109                 if denom == 0:
110                     raise ZeroDivisionError("Divisao por zero.")
111                 node = node / denom
112         return node
113
114     def power(self):
115         if self.current_token[0] in ('NUMBER', 'IDENT', 'LPAREN', 'LBRACK', 'LBRACE'):
116             node = self.factor()
117             if self.current_token[0] == 'POW':
118                 self.eat('POW')
119                 exponent = self.power()
120                 node = node ** exponent
121             return node
122         self.error(f"Fator invalido para potencia: {self.current_token}")
123
124     def expr(self):
125         node = self.term()
126         while self.current_token[0] in ('PLUS', 'MINUS'):
127             op = self.current_token[0]
128             self.advance()
129             if op == 'PLUS':
130                 node = node + self.term()
131             elif op == 'MINUS':
132                 node = node - self.term()
133         return node
134
135     def parse(self):
136         if self.current_token[0] == 'IDENT' and self.pos + 1 < len(self.tokens) and self.tokens[self.pos + 1][0] == 'ASSIGN':
137             name = self.current_token[1]
138             self.eat('IDENT')
139             self.eat('ASSIGN')
140             value = self.expr()
141             self.env[name] = value
142             if self.current_token[0] != 'EOF':
143                 self.error("Tokens extras detectados apos atribuicao.")
144             return value
145
146         result = self.expr()
147         if self.current_token[0] != 'EOF':

```

```
148         self.error("Tokens extras detectados no fim da expressao.")
149     return result
```

5 Conclusão

A arquitetura proposta demonstra viabilidade para execução confiável de expressões matemáticas aninhadas. O uso do padrão de análise descendente recursiva aliada ao encapsulamento léxico por expressões regulares confere modularidade e extensibilidade para futuras implementações, como funções nativas (`sin`, `cos`, `log`) ou blocos de controle estruturados.